

Sharing Conflict Weights Without Partitioning: A Concurrent dom/wdeg Portfolio for Word-Level Crossword Filling

[Author Name(s)]

[Department], [University]

Submitted to IEEE MIT URTC 2026

Abstract—Parallel CSP solvers typically take one of two forms: partition the search space across workers, or run an undivided portfolio of independent restarts. When CSP workers share information at all, prior work treats that sharing as a one-time input to partitioning, gathered once and then spent. This paper describes a different mechanism: a restart portfolio whose workers never partition the problem, but continuously and concurrently update a shared pool of dom/wdeg-style conflict weights throughout the search, with no manager and no synchronization phase. We implement it in *xfill*, a word-level CSP solver for crossword filling, and test it two ways. First, we establish that restart-portfolio search as a class occupies a genuinely different point in the design space than partition-based search, using *orca-solver* (an independently developed partition-based solver) as a concrete representative of the alternative: restart portfolios exploit heavy-tailed search-runtime distributions, so oversubscribing thread count past the physical core count improves *xfill* by up to 65 $\times$ on one hard grid, while the partition-based solver improves only marginally and cannot solve it at all below 14 threads; on 16 real grids both solve, *xfill* is faster on 15, by a geometric mean of 96 $\times$. Partitioning, conversely, gives genuine non-overlapping coverage, a real advantage for proving unsatisfiability. Second, and more directly on point, we isolate the shared-weight mechanism itself – same solver, same grid, only the mechanism toggled – and find a more nuanced result than the architectural comparison above: consistent improvement on a broad corpus of moderate-difficulty grids, but higher variance rather than a reliable win on especially hard instances. We report this honestly rather than selectively, since a negative or mixed result for one’s own mechanism is still evidence. We also report a negative result from the same codebase: five independently plausible letter-level branching heuristics produced numerically identical output on our hardest benchmark grid, because a domain-size precondition meant none of them ever executed – caught only by direct execution-path instrumentation, not by comparing benchmark scores.

I. Introduction

Crossword filling – assigning a dictionary word to every slot in a grid so every crossing letter agrees – is a word-level CSP and a convenient search-research testbed: instances are human-legible, difficulty is controllable directly through grid geometry, and independently developed solvers already exist to compare against. Ginsberg’s *Dr.Fill* [1] established crossword filling as a serious weighted-CSP application over a decade ago.

This paper is not about crossword filling. It uses crossword filling as a setting to answer one question

about parallel constraint search: when multiple workers run concurrently, is there value in sharing soft heuristic statistics – information that biases search order without asserting anything logically – continuously, without ever partitioning the problem those workers are solving? Section II shows this combination does not exist in prior work; Section III implements it; Section V measures its effect, both in isolation and against the broader architectural backdrop that makes it worth measuring at all.

Contributions: (1) a concurrent, decentralized conflict-weight-sharing mechanism for restart-portfolio CSP search, positioned precisely against the closest prior work; (2) a reproducible five-solver testbench, checked into the project repository, that separates two claims – restart-portfolio search as an architectural class versus this specific mechanism within it – rather than conflating them; and (3) a methodological negative result about confirming a heuristic under test actually executes before trusting a benchmark comparison of it.

II. Background and Related Work

A CSP assigns values to variables from finite domains subject to constraints; systematic backtracking assigns one variable at a time, propagating consistency and backtracking on a wipeout. The dom/wdeg heuristic [4] selects the variable with smallest domain-size-to-weighted-degree ratio, where a constraint’s weight increases on every wipeout it causes. Randomized backtracking exhibits heavy-tailed runtime [2], [3]: restarting periodically eliminates most of the tail and is standard in SAT and CSP solvers alike.

Two dominant parallel strategies exist. Portfolio approaches run independent searches over the whole problem concurrently; SAT portfolios such as *ManySAT* [6] additionally share learned clauses between workers without partitioning. Partitioning approaches divide the search space so each worker explores a disjoint region, giving non-overlapping coverage useful for proving unsatisfiability.

Three prior CSP results sit near, but not on, the point in this design space we describe. Grimes and Wallace [5] accumulate dom/wdeg-style weights from short restarted probes of a single sequential solver, before the real search begins – never concurrent. Ehlers and Stuckey [7] parallelize a lazy-clause-generation solver by sharing

nogoods and trail state between workers – concurrent, but sharing hard, logically derived facts, and requiring clause-learning machinery most CSP solvers do not have. Yun and Epstein’s ELF and SPREAD [8], [9] are the closest: a manager races workers under a plain dom/wdeg solver, then averages their reported weights once each exhausts a budget, and uses the average to choose how to partition the problem for a subsequent phase. The authors state this directly: “the primary purpose of SPREAD’s portfolio phase is to glean information to support search space splitting, not to solve P ” [9], and separately that “most portfolio-based methods for CSPs do not share information” [9] at all outside their framework. A comprehensive 2018 survey [11] documents no CSP work sharing heuristic statistics continuously among concurrent, non-partitioning workers; its own taxonomy has no category for it. Embarrassingly parallel search [10], a widely used later technique, decomposes into disjoint subproblems with “almost no communication” at all – a different point again.

The mechanism in Section III occupies the gap these leave: concurrent, unlike Grimes and Wallace; soft, unlike Ehlers and Stuckey; and never used to partition, unlike Yun and Epstein.

III. System Architecture

xfill parses a grid into slots (open runs ≥ 2 cells) and crossings (cell-level intersections). Its sequential core performs dom/wdeg-style branching – selecting the slot minimizing domain size over accumulated crossing weight – then propagates letter constraints via an AC-3-style queue; a domain wipeout increases the responsible crossing’s weight.

SolveParallel runs N independent restart workers, each with its own seed and private, decaying crossing-weight table, plus one worker dedicated to a single uninterrupted exhaustive search (unlimited_budget) that guarantees eventual completion independent of restart variance. Any worker’s sound negative result cancels the others.

Concurrent conflict-weight sharing. Alongside each worker’s private table, SolveParallel maintains one array of plain atomic counters, one per crossing, shared by every restart worker except the dedicated exhaustive one. Every worker increments a crossing’s counter on wipeout and reads every counter when scoring candidates. The counters are plain, order-independent atomic increments rather than a decaying scheme: a decaying normalizer depends on the exact sequence of prior updates and is race-free only with a single writer, while concurrent writers need an update rule that does not depend on order. The dedicated exhaustive worker is excluded because its entire value is one undisturbed trajectory; wiring it in measurably regressed the one grid in our corpus it uniquely solved. Two correctness properties were required before any of the resulting numbers meant anything: single-threaded calls stay byte-identical to the unshared baseline,

gated on thread count > 1 ; and ThreadSanitizer, run against real multi-threaded solves, reports no data races, consistent with every update being independent of order. In earlier development-time testing across a broad corpus of moderate-difficulty grids (documented at commit level in this repository’s docs/design.md), this mechanism cut solve time broadly – one grid from 12.5s to 0.7s, another from 1.2s to 0.1s, a third newly solved within its time cap – a finding we revisit against a harder, targeted ablation in Section V-G.

IV. A Negative Result: Branching on the Wrong Thing

This section is a methodological aside from the same codebase, not a direct test of the mechanism above; we include it because it shaped how we validate every other result in this paper. Motivated by a 7×7 grid known to be exceptionally hard for restart-based search (Section V-B), we implemented five letter-level branching variants – two minimum-remaining-value scoring rules, a faithful reimplement of a partition-based solver’s own candidate scoring (read from its published source for reference), and a reversed scan order – reasoning that a smaller per-node branching factor should help on wide-open grids.

All five produced numerically identical node and back-track counts. Five independently reasonable heuristics agreeing to the exact node is a stronger signal than “the heuristics didn’t help,” so we checked rather than reported a null result. A plain counter placed directly in the branch-selection code, tracking how often the large-domain branch these heuristics govern actually executed, showed it firing zero times across more than 17,000 real search nodes: propagation from the grid’s pre-filled letters narrowed every domain below the branch’s activation threshold within the first few assignments. All five heuristics were unreachable code the entire time we compared them, which is exactly why comparing their benchmark scores could not have distinguished them. The actual bottleneck, found only after ruling out the large-domain branch, was the small-domain branch’s fixed word ordering; extending the solver’s existing restart-randomization flag to that branch produced the improvements reported in Section V-B. The reusable point: an A/B comparison of branching heuristics is only informative once the branch under test is confirmed, by direct instrumentation, to actually execute.

V. Experiments

A. Setup

The experiments below test two distinct claims, and we keep them in separate subsections rather than blending them. Sections V-B–V-F test whether restart-portfolio search, as an architectural class, occupies a genuinely different point in the design space than partition-based search – using orca-solver as a concrete, independently developed representative of that alternative, and two

TABLE I

Thread count vs. wall time on a known-hard 7×7 grid (single sample; a partition-based solver’s published single-threaded baseline on this grid is 201.7s).

Threads	Wall time (s)
1	166.7
14	63.0
28	104.6
42	85.6
56	112.4

unmodified naive backtrackers to show the comparison is not trivial. This establishes why the mechanism is worth building at all. Section V-G then isolates the mechanism itself, holding architecture fixed, which is this paper’s actual contribution.

Solvers. xfill (this work); orca-solver, a partition-based parallel solver treated as a black box; ingrid_core, a single-threaded reference solver; and, as naive baselines with no restarts, no learned heuristics, and no parallelism, crossword-composer (a static-constraint-order Rust backtracker) and savin_crossword (a textbook AC-3-plus-backtracking Python implementation). All five run through a single fixed harness checked into this project’s repository (benchmarks/urtc2026_testbench/), against a dictionary derived deterministically from the repository’s own freely available word list, with a fixed random seed and a uniform 30-second cap – reproducible by re-running run_benchmark.py.

Grid set. A size-graded curated set (5×5 through 21×21) plus a fixed-seed random sample of 15×15 grids from a real, previously scraped corpus, all committed to the repository.

B. Restart Portfolios Exploit Added Parallelism

Restart-portfolio search should benefit from more independent workers even past the physical core count, because heavy-tailed runtime means each additional worker is another draw that might get lucky early. Table I tests this directly on a known-hard 7×7 grid (two pre-filled letters, no blocked cells), oversubscribing threads well past this machine’s 14 physical cores. Wall time improves through 42 threads before contention erodes further gains at 56; every configuration beats the 201.7s single-threaded baseline an independently developed partition-based solver reports on the same grid, which serves here only as an external difficulty anchor, not a target being chased.

C. Is This a Property of the Architecture, or One Grid?

Fig. 1 repeats the oversubscription experiment on three more grids, at 1 through 42 threads, for both xfill and orca-solver, to check whether Table I’s result is a general property of restart portfolios or an artifact of one grid. It is not one story: on grid_120, xfill’s wall time falls from 27.8s at 1 thread to 0.43s at 42 – a $65\times$ improvement – while

Thread-count scaling: restart portfolio vs. partitioned search (dashed line = physical core count)

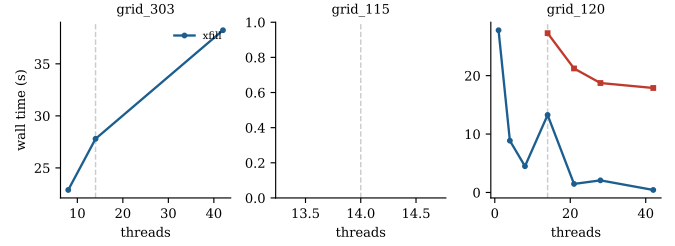


Fig. 1. Wall time vs. thread count, xfill vs. orca-solver, on three grids known to be hard for restart-based search (45s cap; orca-solver missing from a panel means every one of its 7 thread counts timed out). grid_120 is the clearest case: xfill improves roughly $65\times$ from 1 to 42 threads while orca-solver stays flat and cannot solve at all below 14.

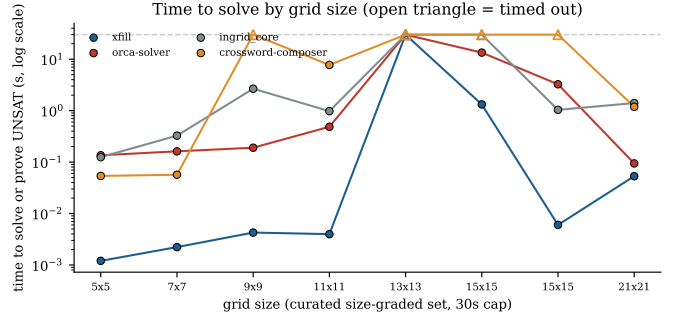


Fig. 2. Time to solve or prove UNSAT by grid size, curated set, 30s cap (log scale; open triangle = timed out). A solved/not-solved view of this same data would show xfill, orca-solver, and ingrid_core essentially tied, since all three succeed on the same grids here – the actual difference is that xfill’s line sits one to three orders of magnitude below the others at every size except 13×13 , where every solver times out together.

orca-solver never solves it below 14 threads and only creeps from 27.3s to 17.9s from 14 to 42; on grid_303, orca-solver fails to solve it at any of the 7 thread counts tested, while xfill solves it (noisily, not monotonically) at 8, 14, and 42 threads; on grid_115, neither solver succeeds at any thread count – not every grid favors either architecture, and we report that rather than omit it. Where the pattern does hold, it holds because absorbing extra parallelism without restructuring the search is what a non-partitioning restart portfolio is built to do; a partition-based design here does not exhibit that capacity to nearly the same degree.

D. Naive Backtracking Is Not a Baseline Worth Beating

Across all 20 grids, xfill solved 17 (1 correct UNSAT, 2 timeouts), orca-solver 16 (1 UNSAT, 3 timeouts), ingrid_core 12, crossword-composer 5, and savin_crossword 0 – it timed out on every grid, including the trivial 5×5 , and so never appears in Fig. 2. That absence is itself informative: crossword-composer, which does appear intermittently, fails on individual instances despite having no dependence on dictionary size, while savin_crossword’s failure is not about grid size at all – its least-constraining-

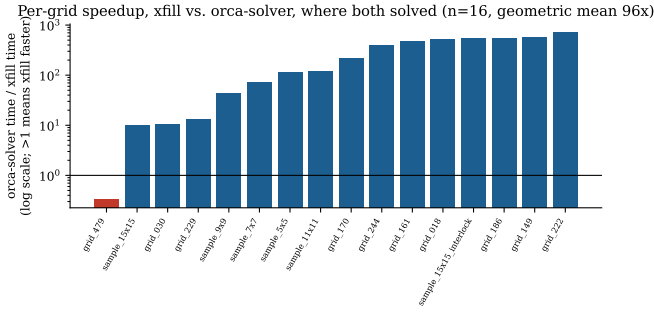


Fig. 3. Per-grid speedup (orca-solver time / xfill time, log scale) on every grid in the full 20-grid sample both solvers solved. Bars above the line mean xfill is faster; only one of 16 falls below it.

value step recomputes an $O(|\text{domain}|^2)$ score from scratch at every node, which our $\sim 184,000$ -word dictionary makes prohibitive regardless of size. Both are real, unmodified, independently authored solvers. The point for this paper’s thesis is narrower than “xfill wins”: it is that restarts, adaptive heuristics, and parallelism are not optional refinements at the grid and dictionary sizes crossword filling actually requires, which is the premise the rest of this section, and the mechanism this paper contributes to it, depends on.

E. Aggregate Speed Advantage Where Both Solve

Fig. 3 quantifies the restart-portfolio/partition comparison across the full sample rather than the curated subset alone: of the 16 grids (of 20) both xfill and orca-solver solve, xfill is faster on 15, with a median speedup of $171\times$ and a geometric mean of $96\times$, driven by several 15×15 grids where xfill finishes in single-digit milliseconds against 3–13 seconds. The one exception, grid_479, runs the other way (orca-solver $\sim 3\times$ faster). We report it rather than omit it: a lopsided aggregate is still not a universal one, consistent with grid_115 in Section V-C. This sample also happens to be too easy to separate the two architectures on success alone – both solve 16 of 20, agree on one correct UNSAT proof, and are jointly stuck on 2, with orca-solver never uniquely solving a grid xfill cannot – which is exactly why Sections V-C and V-G use a harder, pre-existing grid list to find where the architectures actually diverge, rather than relying on this sample alone.

F. Exhaustive Case Study: Proving, Not Just Finding

Every experiment so far tests finding a solution quickly. Partitioning’s genuine non-overlapping coverage (Section II) should instead matter for proving none exists. To test this directly, we constructed by hand a grid designed to be maximally constrained – five mutually crossing 15-letter entries, verified cell-by-cell against both solvers’ own parsers – and ran both to completion with no time limit. Both independently concluded no solution exists: xfill exhausted 2.16 billion nodes over 791 restarts in 76 minutes; orca-solver reported search exhausted after

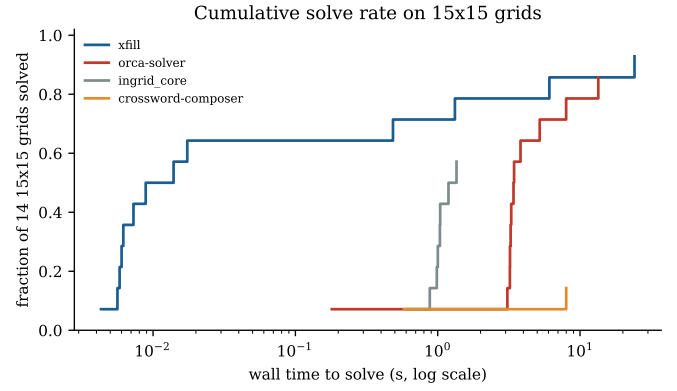


Fig. 4. Cumulative fraction of real 15×15 grids solved by wall time (log scale), 30s cap. The point is the horizontal gap between curves, not the final height: xfill’s curve is shifted roughly two orders of magnitude left of ingrid_core’s and orca-solver’s, meaning it reaches the same solved fraction far sooner across the entire distribution, not only in a best-case tail.

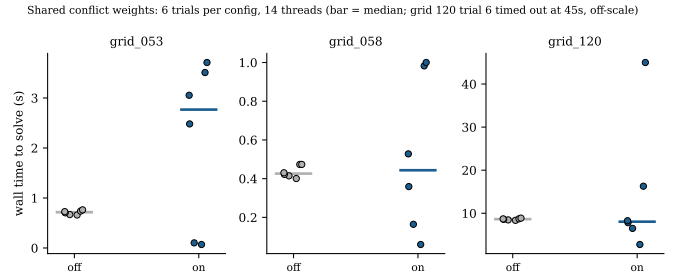


Fig. 5. Shared-conflict-weight ablation: 6 trials per configuration, 14 threads, on three grids from the same known-hard list as Fig. 1 that are fast enough to repeat. Grey = mechanism off, blue = on (this paper’s default). Bars mark the median; the spread is the finding.

enumerating 224 partitions in 25 minutes, roughly $3\times$ faster. This is the one result in this paper where the partition-based architecture, not the restart portfolio, has the structural advantage, consistent with Section II’s framing of what partitioning is actually for.

G. Isolating the Mechanism: A Shared-Weight Ablation

Everything above tests xfill’s architecture against a different codebase’s. This section instead tests xfill against itself, with one environment variable flipped, isolating this paper’s actual contribution from every other difference: same solver, same grid, same dictionary, same 14 threads, `XFILL_DISABLE_SHARED_WEIGHTS` on and off, 6 trials each, on three grids from the known-hard list fast enough to repeat within a reasonable budget.

Fig. 5 does not show a uniform win for the mechanism on this specific, especially-hard grid subset, and we report that plainly rather than around it. On grid_053, the unshared baseline is both faster on the median (0.72s vs. 2.77s) and far tighter (range 0.66–0.76s vs. 0.07–3.71s). On grid_058, the medians are close but shared weights again show wider spread. On grid_120, shared weights reach a

lower best case (2.75s vs. 8.36s minimum) but also produce the only outright timeout of the eighteen trials shown. Set against Section III’s broader-corpus finding – a consistent win across moderate-difficulty grids – the synthesis is that this mechanism’s benefit is regime-dependent: it helps on typical grids by redistributing which crossings each worker deprioritizes, but on grids already at the extreme difficulty end it trades a tight, well-behaved variance for a wider one that is not reliably better on net. We have no evidence for a clean win here, so we do not claim one; a negative or mixed ablation reported honestly is itself consistent with Section IV’s stance on what makes a heuristic comparison trustworthy.

VI. Discussion

The architectural comparison and the mechanism ablation point to the same underlying cause. Restart portfolios benefit from heavy-tailed runtimes: more independent draws raise the chance of an early lucky one, which is why oversubscribing thread count helped in Table I and Fig. 1, and why the aggregate speed advantage in Fig. 3 is a finding-oriented result rather than a general one. Partitioned search instead gives genuine non-overlapping coverage, which is what Section V-F’s proof of unsatisfiability actually required; restart-based search approximates that only through its dedicated exhaustive worker, at the cost of forgoing restarts entirely for it. The same heavy-tailedness explains Fig. 5’s mixed result: shared conflict weights change which draw from the tail each worker takes, which can as easily land on a worse draw as a better one on any single hard instance, and only nets out as an improvement once averaged over a broader, more typical corpus. The architectural comparison and the mechanism ablation are not in tension; they describe the same phenomenon at two different points on the difficulty spectrum, and the paper’s contribution is the latter, not the former.

VII. Limitations and Conclusion

All timing results are from one 14-core machine and one dictionary derived from a specific freely available word list; neither the thread-scaling numbers nor Section V-F’s unsatisfiability result should be read as hardware- or dictionary-independent claims. Both principal solvers are actively developed software; exact commit hashes are recorded in the testbench README for reproducibility.

This paper’s contribution is the mechanism in Section III: concurrent conflict-weight sharing across a restart portfolio that never partitions, occupying a gap that three adjacent lines of prior work leave open. The architectural comparison in Section V is supporting evidence for why that gap is worth occupying, not the paper’s subject – restart portfolios and partitioned search are suited respectively to finding solutions quickly and to proving none exist, a pattern independent of any one solver. The mechanism itself, tested in isolation, is more modest than

the architecture it sits inside: a real win on typical grids, a wash or worse on the hardest ones, reported as measured rather than as hoped. We also reported a negative result – five heuristics rendered indistinguishable by a precondition that kept them from ever executing – as a reusable caution for comparing branching heuristics by benchmark score alone.

Acknowledgments

[Add any advisor, funding, or collaborator acknowledgments here.]

References

- [1] M. L. Ginsberg, “Dr.Fill: Crosswords and an Implemented Solver for Singly Weighted CSPs,” *Journal of Artificial Intelligence Research*, vol. 42, pp. 851–886, 2011.
- [2] C. P. Gomes, B. Selman, and H. Kautz, “Boosting Combinatorial Search Through Randomization,” in *Proc. AAAI*, 1998, pp. 431–437.
- [3] C. P. Gomes, B. Selman, N. Crato, and H. Kautz, “Heavy-tailed phenomena in satisfiability and constraint satisfaction problems,” *Journal of Automated Reasoning*, vol. 24, pp. 67–100, 2000.
- [4] F. Boussemart, F. Hemery, C. Lecoutre, and L. Sais, “Boosting systematic search by weighting constraints,” in *Proc. ECAI*, 2004, pp. 146–150.
- [5] D. Grimes and R. J. Wallace, “Sampling Strategies and Variable Selection in Weighted Degree Heuristics,” in *Proc. CP, LNCS* vol. 4741, 2007, pp. 831–838.
- [6] Y. Hamadi, S. Jabbour, and L. Sais, “ManySAT: a Parallel SAT Solver,” *J. Satisfiability, Boolean Modeling and Computation*, vol. 6, pp. 245–262, 2009.
- [7] T. Ehlers and P. J. Stuckey, “Parallelizing Constraint Programming with Learning,” in *Proc. CPAIOR, LNCS* vol. 9676, Banff, Canada, 2016.
- [8] X. Yun and S. L. Epstein, “Adaptive Parallelization for Constraint Satisfaction Search,” in *Proc. SoCS*, 2012, pp. 145–152.
- [9] X. Yun and S. L. Epstein, “A Hybrid Paradigm for Adaptive Parallel Search,” in *Proc. CP, LNCS* vol. 7514, 2012, pp. 720–736.
- [10] J.-C. Régim, M. Rezgui, and A. Malapert, “Embarrassingly Parallel Search in Constraint Programming,” *Journal of Artificial Intelligence Research*, vol. 57, pp. 421–464, 2016.
- [11] I. P. Gent, I. J. Miguel, P. W. Nightingale, C. McCreesh, P. Prosser, N. Moore, and C. Unsworth, “A Review of Literature on Parallel Constraint Solving,” *Theory and Practice of Logic Programming*, vol. 18, no. 5–6, pp. 725–758, 2018.